

Phase 2 du projet

MTH8211

Oussama Mouhtal, Yasmine Amami

Changement du Modèle et du code source

Dans cette phase du projet, nous avons décidé de ne pas utiliser le modèle `shallowwater1D`, car il s'agit d'un modèle linéaire. De plus, nous avons rencontré des problèmes d'implémentation : il fallait s'appuyer sur un ancien code, écrit de manière obsolète, avec une implémentation inefficace de l'approximation de la Hessienne, pourtant essentielle dans ce projet. Pour ces raisons, nous avons choisi de recoder notre propre solution *from scratch*, en utilisant le modèle **Lorenz 95**.

Modèle Lorenz 95

Le **modèle de Lorenz-96** est un système dynamique simplifié souvent utilisé comme modèle mathématique jouet pour étudier certaines propriétés des systèmes chaotiques et des phénomènes en météorologie.

Le modèle de Lorenz-96 décrit un ensemble de n variables continues, généralement notées $x_1, x_2, \dots, x_n$. Chacune de ces variables représente une grandeur qui évolue dans le temps en fonction de ses interactions avec les autres variables. La dynamique des variables dans le modèle de Lorenz-96 est gouvernée par les équations du type :

$$\frac{dx_i}{dt} = -x_{i-2}x_{i-1} + x_{i-1}x_{i+1} - x_i + F, \quad 1 \leq i \leq n$$

où l'on impose des *conditions aux limites périodiques*, à savoir : $x_{-1} = x_{n-1}$, $x_0 = x_n$ et $x_{n+1} = x_1$. La constante F est un paramètre du modèle, généralement fixé à $F = 8$.

- Le système d'équations 195 est intégré en utilisant un schéma Runge Kutta explicité, le modèle obtenu est appelée `traj`.

Test des opérateur tangentielle et leur adjoint

Les opérateurs tangentiels (TLM, Tangent Linear Model) et leurs adjoints (ADJ) jouent un rôle central dans les méthodes de moindres carrées non linéaires notamment en assimilation de données. Afin de garantir la validité des implémentations numériques, il est essentiel de vérifier la cohérence entre ces opérateurs et de s'assurer qu'ils respectent les propriétés mathématiques attendues. Dans cette section, nous présentons une série de tests visant à valider la consistance entre les opérateurs tangentiels et leurs adjoints, ainsi que leur précision par rapport à des différences finies, tant pour le modèle `traj` que pour l'opérateur d'observation `hop` et leur composition `gop`.

```
n = 40
nt = 10
dt = 0.01
F = 8.0
model = Lorenz95Model(F, dt)
```

```

# Pour les tests de l'observation
sigmaR = 0.5
total_space_obs = 10
total_time_obs = 5
space_inds_obs = round(Int, range(1, n; length=total_space_obs))
time_inds_obs = round(Int, range(1, nt-1; length=total_time_obs))
obs = ObsOperator(sigmaR, space_inds_obs, n, time_inds_obs, nt, model)

@testset "TLM-ADJ Consistency (Model)" begin
    x = randn(n)
    dx = randn(n)
    a = randn(n)

    y1 = tlm_traj(model, x, dx, nt)
    y2 = ad_traj(model, x, a, nt)

    inner1 = dot(y1, a)
    inner2 = dot(dx, y2)

    @test inner1 inner2 atol=1e-7
end

@testset "TLM vs Finite Differences (Model)" begin
    x = randn(n)
    dx = randn(n)
    = 1e-6

    fx = traj(model, x , nt)
    fx_perturbed = traj(model, x .+ .* dx, nt)
    fd_approx = (fx_perturbed .- fx) ./

    tlm_val = tlm_traj(model, x, dx, nt)

    @test fd_approx tlm_val atol=10*
end

@testset "TLM-ADJ Consistency (Opérateur d'observation)" begin
    x = randn(n)
    dx = randn(n)
    ay = randn(length(space_inds_obs))

    y1 = tlm_hop(obs, dx)
    y2 = adj_hop(obs, ay)

    inner1 = dot(y1, ay)
    inner2 = dot(dx, y2)

    @test inner1 inner2 atol=1e-7
end

@testset "TLM vs Finite Differences (Opérateur d'observation)" begin
    x = randn(n)
    dx = randn(n)

```

```

    = 1e-6

    fx = hop(obs, x)
    fx_perturbed = hop(obs, x .+ .* dx)
    fd_approx = (fx_perturbed .- fx) ./

    tlm_val = tlm_hop(obs, dx)

    @test fd_approx tlm_val atol=10*
end

@testset "TLM vs Finite Differences (G = H o M Composition du model et Obs)" begin
    x = randn(n)
    dx = randn(n)
    = 1e-6

    fx = gop(obs, x)
    fx_perturbed = gop(obs, x .+ .* dx)
    fd_approx = (fx_perturbed .- fx) ./

    tlm_val = tlm_gop(obs, x, dx)

    @test fd_approx tlm_val atol=10*
end

@testset "gop: TLM-ADJ Consistency (G operator)" begin
    xt = randn(n)
    dx = randn(length(xt))
    a = randn(length(space_inds_obs) * length(time_inds_obs))

    y1 = tlm_gop(obs, xt, dx)
    y2 = adj_gop(obs, xt, a)

    inner1 = dot(y1, a)
    inner2 = dot(dx, y2)

    @test inner1 inner2 atol=1e-7
end

```

```

Test Summary: | Pass Total Time
TLM-ADJ Consistency (Model) | 1 1 0.5s
Test Summary: | Pass Total Time
TLM vs Finite Differences (Model) | 1 1 0.0s
Test Summary: | Pass Total Time
TLM-ADJ Consistency (Opérateur d'observation) | 1 1 0.0s
Test Summary: | Pass Total Time
TLM vs Finite Differences (Opérateur d'observation) | 1 1 0.0s
Test Summary: | Pass Total Time
TLM vs Finite Differences (G = H o M Composition du model et Obs) | 1 1 0.0s
Test Summary: | Pass Total Time
gop: TLM-ADJ Consistency (G operator) | 1 1 0.1s

```

```
Test.DefaultTestSet("gop: TLM-ADJ Consistency (G operator)", Any[], 1, false, false, true, 1.7540140197)
```

Code 4Dvar

- La fonction `fourdvar` implémente une assimilation de données variationnelle **4D-Var incrémental**, appliqué au modèle dynamique de Lorenz 95. Elle vise à estimer l'état initial d'un système à partir d'une connaissance a priori et d'observations bruitées dans le temps.
- L'approche utilisée repose sur la méthode de **Gauss-Newton** pour la résolution d'un problème d'optimisation non linéaire. À chaque itération externe, un sous-problème quadratique est résolu par une méthode de Krylov, permettant de comparer différents solveurs itératifs (MINRES, CG, MINRES-QLP, etc.).
- A chaque utilisation d'un **sous-solveur**, on affiche les informations suivantes :
 - Le **nombre d'itérations** nécessaires pour converger
 - Le **résidu initial** ($\|r_0\|$)
 - Le **résidu final** ($\|r_f\|$)Ces données sont reportées à intervalles réguliers pendant les itérations de **Gauss-Newton**.

Remarque : les critères d'arrêt utilisés pour les sous-solveurs sont ceux définis par défaut.

- Enfin, un tableau récapitulatif présente les statistiques globales pour chaque sous-solveur :
 - Le **nombre total d'itérations Gauss-Newton** effectuées pour satisfaire le critère d'arrêt
 - La **norme du gradient** final
 - La **valeur de la fonction coût non linéaire** finale
 - L'**erreur relative** par rapport à la vérité (norme relative de l'erreur)
- Les critères d'arrêt de la boucle externe sont définis par défaut comme suit :

`max_outer = 25, atol_grad = 10-10, rtol_grad = 10-10`

```
"""
    fourdvar(n, nt, dt, F, sigmaR, total_space_obs, total_time_obs, sigmaB)

Réalise une assimilation de données 4D-Var incrémentale sur le modèle de Lorenz 95
en utilisant différents sous-solveurs itératifs pour la résolution du problème quadratique.

# Arguments
- `n::Int` : nombre de variables d'état du modèle (taille de l'espace d'état).
- `nt::Int` : nombre de pas de temps.
- `dt::Float64` : pas de temps d'intégration.
- `F::Float64` : paramètre de forçage du modèle Lorenz 95.
- `sigmaR::Float64` : écart-type de l'erreur d'observation.
- `total_space_obs::Int` : points observés dans l'espace.
- `total_time_obs::Int` : nombre d'observations dans le temps (fenêtre d'observation).
- `sigmaB::Float64` : écart-type de l'erreur de l'information à priori.

# Description

La fonction :
1. Initialise le modèle de Lorenz 95.
2. Génère un état de vérité `xt` par spin-up.
3. Définit une information à priori `xb` comme une perturbation gaussienne de `xt`.
4. Simule des observations bruitées `y` à partir de `xt`.
5. Résout le problème 4D-Var incrémental par la méthode de Gauss-Newton avec différents sous-solveurs d
6. Retourne les résultats pour analyse.

# Sous-solveurs testés
```

```

- `minres_qlp`
- `minres`
- `cg_lanczos`
- `cg`
- `minares`

# Retour
- `xt` : vérité terrain simulée.
- `xb` : information à priori.
- `y` : observations synthétiques.
- `iterates_GN` : dictionnaire contenant les analyses obtenues avec chaque sous-solveur.
- `model` : instance du modèle `Lorenz95Model`.
- `time_inds_obs` : indices temporels des observations.

"""
function fourdvar(n, nt, dt, F, sigmaR, total_space_obs,
total_time_obs, sigmaB, max_outer = 25,
atol_grad = 1e-10,
rtol_grad = 1e-10)
# Model
model = Lorenz95Model(F, dt)

# Observations

space_inds_obs = round.(Int, range(1, n; length=total_space_obs))
time_inds_obs = round.(Int, range(1, nt-1; length=total_time_obs))
obs = ObsOperator(sigmaR, space_inds_obs, n, time_inds_obs, nt, model)
R = RMatrix(sigmaR)

# Background
B = BMatrix(sigmaB, n)

# Spin-up
xt = 3.0 .* ones(n) .+ randn(n)
xt = traj(model, xt, 5000)

# Background state
xb = xt .+ randn(n) .* sigmaB

# Observations
y = generate_obs(obs, xt)

function nonlinear_funcval(x, y, obs, xb, B, R)
    eo = y .- gop(obs, x)
    eb = x .- xb
    return eb'* invdot(B, eb) + eo'*invdot(R, eo)
end

function quadratic_funcval(x, dx, y, obs, xb, B, R)
    eo = gop(obs, x) .- y .+ tlm_gop(obs, x, dx)
    eb = x .- xb .+ dx
    return dot(eb, invdot(B, eb)) + dot(eo, invdot(R, eo))

```

```

end

xa = copy(xb)

iterates_GN = Dict{String, Vector{Float64}}{ }

# Initialisation des itérées de Gauss-Newton
for name in ["minres_qlp", "minres", "cg_lanczos", "cg", "minares"]
    iterates_GN[name] = copy(xa)
end

row_names = vcat("Outer-Iter", "||grad||", "|f(x_k)|", "||err||")
global stats_GN_table = row_names
for name in ["minres_qlp", "minres", "cg_lanczos", "cg", "minares"]
    iterates_GN[name] = copy(xa)
    nb_iter = 0
    b = zeros(n)
    normb = normb0 = 0.0
    println("Résolution su problème avec le sous-solveur $name")
    global stats_subsolver = vcat("Inner-Iter", "||r0||", "||rf||")
    header_subsolver = [""]
    for iter_outer in 1:max_outer
        d = misfit(obs, y, iterates_GN[name])
        b = invdot(B, xb .- iterates_GN[name]) .+ adj_gop(obs, iterates_GN[name], invdot(R, d))

        if iter_outer == 1
            normb = normb0 = norm(b)
        else
            normb = norm(b)
        end

        A = Hessian4DVar(obs, R, B, iterates_GN[name])
        solver = getproperty(Krylov, Symbol(name))
        dx, stats = solver(A, b, history=true)
        iterates_GN[name] .+= dx
        nb_iter += 1
        if normb < atol_grad + rtol_grad*normb0
            break
        end
        if iter_outer%5 == 0 || iter_outer == 1
            push!(header_subsolver, "GN iter $iter_outer")
            global stats_subsolver = hcat(stats_subsolver, vcat(stats.niter, stats.residuals[1], stats.
        end

    end

    pretty_table(stats_subsolver, header=header_subsolver)

    res_final = norm(b)

```

```

    fval = nonlinear_funcval(iterates_GN[name], y, obs, xb, B, R)
    err_xt = norm(xt - iterates_GN[name]) / norm(xt)
    global stats_GN_table = hcat(stats_GN_table, vcat(nb_iter, res_final, fval, err_xt))
end

println("statistiques sur GN en utilisons différents sous-solveurs")
header = ["", "minres_qlp", "minres", "cg_lanczos", "cg", "minares"]
pretty_table(stats_GN_table, header=header)
    return xt, xb, y, iterates_GN, model, time_inds_obs
end

```

Main.Notebook.fourdvar

Expérience avec $n = 100$ et $m = 50$

On observe que tous les sous-solveurs convergent en un temps quasi-similaire, sauf **minares**, qui nécessite un nombre d'itérations significativement plus faible que les autres. Cela peut être dû aux critères d'arrêt basés sur le A -résidu, ce qui se reflète sur les résidu, où le résidu prosuit par **minares** est légèrement différente.

Cependant, l'erreur relative par rapport à la vérité reste quasiment identique à celle des autres sous-solveurs, compte tenu du nombre de chiffres affichés.

Enfin, on remarque sur les trajectoires en partant avec différentes solutions produites sont visuellement presque identiques.

```

n = 100
nt = 10
dt = 0.025
F = 8.0
sigmaR = 0.1
total_space_obs = 10
total_time_obs = 5 # Le nombre d'observation m = total_space_obs * total_time_obs
sigmaB = 0.8

xtruth, xback, y, iterates_GN, model, time_inds_obs = fourdvar(n, nt, dt, F, sigmaR,
total_space_obs, total_time_obs, sigmaB);

```

Résolution su problème avec le sous-solveur minres_qlp

	GN iter 1	GN iter 5	GN iter 10	GN iter 15	GN iter 20
Inner-Iter	48.0	49.0	34.0	23.0	21.0
r0	2154.61	0.124209	0.00120382	3.79485e-5	1.93075e-6
rf	2.97791e-5	2.98103e-9	1.00277e-8	1.45235e-8	3.25168e-9

Résolution su problème avec le sous-solveur minres

	GN iter 1	GN iter 5	GN iter 10	GN iter 15	GN iter 20
Inner-Iter	48.0	48.0	33.0	23.0	22.0
r0	2154.61	0.124209	0.00120382	3.79485e-5	1.93077e-6
rf	1.60294e-5	1.63195e-8	1.29861e-8	1.47589e-8	1.02443e-8

Résolution su problème avec le sous-solveur cg_lanczos

	GN iter 1	GN iter 5	GN iter 10	GN iter 15	GN iter 20
Inner-Iter	51.0	49.0	35.0	25.0	22.0
r0	2154.61	0.124209	0.00120382	3.79487e-5	1.93081e-6
rf	1.04389e-5	3.00505e-9	7.9066e-9	4.75812e-9	9.58449e-9

Résolution su problème avec le sous-solveur cg

	GN iter 1	GN iter 5	GN iter 10	GN iter 15	GN iter 20
Inner-Iter	49.0	48.0	35.0	25.0	21.0
r0	2154.61	0.124209	0.00120382	3.79487e-5	1.93078e-6
rf	8.34593e-6	7.64919e-9	7.51605e-9	4.73628e-9	1.31726e-8

Résolution su problème avec le sous-solveur minares

	GN iter 1	GN iter 5	GN iter 10	GN iter 15	GN iter 20
Inner-Iter	36.0	44.0	38.0	23.0	22.0
r0	2154.61	0.124314	0.00120687	3.8176e-5	1.94377e-6
rf	0.00837387	1.78596e-7	1.26809e-8	1.43615e-8	3.04789e-9

statistiques sur GN en utilisons différents sous-solveurs

	minres_qlp	minres	cg_lanczos	cg	minares
Outer-Iter	24.0	24.0	24.0	24.0	24.0
grad	1.81422e-7	1.815e-7	1.80947e-7	1.81147e-7	1.85381e-7
f(x_k)	47.8161	47.8161	47.8161	47.8161	47.8161
err	0.161524	0.161524	0.161524	0.161524	0.161524

```

x_truth_full = Vector{Vector{Float64}}{ }()
push!(x_truth_full, xtruth)
for _ in 1:nt
    global xtruth = traj(model, xtruth, 1)
    push!(x_truth_full, xtruth)
end

# Trajectoire background

x_b_full = Vector{Vector{Float64}}{ }()
push!(x_b_full, xback)
for _ in 1:nt
    global xback = traj(model, xback, 1)
    push!(x_b_full, xback)
end

trajectories = Dict{String, Vector{Vector{Float64}}}{ }()
# Trajectoire analysée
for name in ["minres_qlp", "minres", "cg_lanczos", "cg", "minares"]
    trajectories[name] = Vector{Vector{Float64}}{ }()
end

```

```

    global xanalysis = copy(iterates_GN[name])
    push!(trajectories[name], xanalysis)
    for _ in 1:nt
        global xanalysis = traj(model, xanalysis, 1)
        push!(trajectories[name], xanalysis)
    end
end

# Observations
y_obs = reshape(y, total_space_obs, total_time_obs)

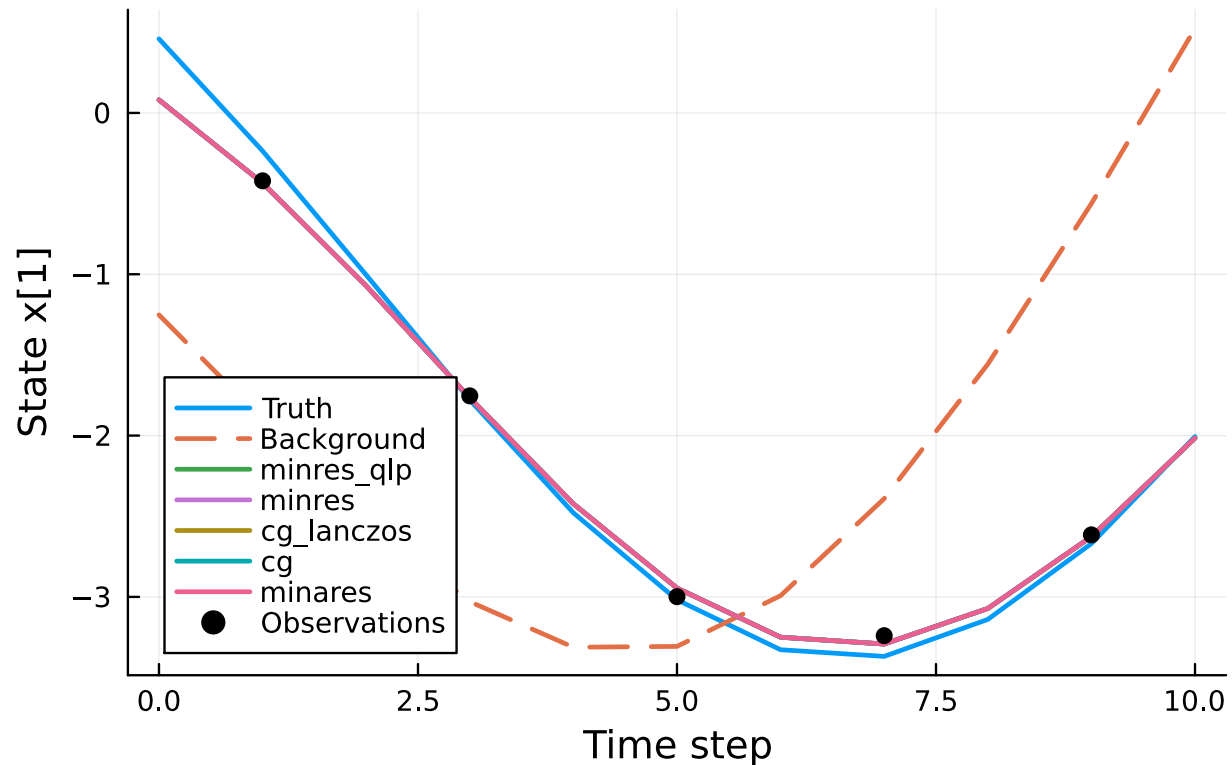
# Plot d'un état : par exemple la variable n°1 en fonction du temps
idx = 1 # index de la variable à tracer

times = 0:nt
gr()
plot(times, [x[idx] for x in x_truth_full], label="Truth", lw=2)
plot!(times, [x[idx] for x in x_b_full], label="Background", lw=2, ls=:dash)
for name in ["minres_qlp", "minres", "cg_lanczos", "cg", "minares"]
    plot!(times, [x[idx] for x in trajectories[name]], label=name, lw=2)
end
scatter!(time_inds_obs, y_obs[1, :], label="Observations", ms=4, c=:black)
xlabel!("Time step")
ylabel!("State x[$idx]")
savefig("assimilation_result.pdf")

"/Users/mouhtal/DataAssim.jl/assimilation_result.pdf"

```

Trajectoires générés par différents analyses



Prochaine travaux

- Afficher davantage de statistiques sur les différents solveurs afin de pouvoir tirer des conclusions sur leur efficacité lorsque la taille du problème augmente.
- Toutes les méthodes utilisées sont actuellement appliquées aux équations normales (quadratique); les prochaines étapes consistent à résoudre les sous-problèmes de moindres carrés en utilisant `lsqr` et `lsmr`.
- Utiliser des préconditionneurs pour améliorer la résolution des équations normales.

Lien Github

- <https://github.com/Oussamamouhtal/DataAssim.jl>
- <https://github.com/Oussamamouhtal/DataAssim.jl/blob/phase2/rapportphase2.qmd> (lien vers le rapport)
- Le modèle Lorenz95 est implémenté dans "src/lorenz95.jl"
- Les opérateurs nécessaires sont implémentées dans "src/operators.jl"